

Taller 02: Fork()

SISTEMAS OPERATIVOS

INTEGRANTES:

ANDRES DIAZ



Pontificia Universidad
JAVERIANA
Colombia

2025

1. Objetivos

- Desarrollar un programa en C que implemente la creación jerárquica de procesos mediante `fork()` y la comunicación entre ellos utilizando pipes.
- Consolidar el uso de paso de argumentos al proceso principal, manejo de archivos, memoria dinámica y sincronización entre procesos.

2. Descripción del Problema

El programa debe recibir como argumentos dos archivos de texto y la cantidad de enteros que contiene cada uno, con el siguiente formato:

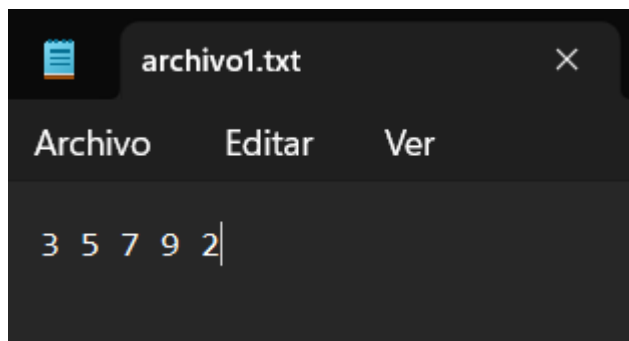
`./taller_procesos N1 archivo00 N2 archivo01`

```
estudiante@NGEN154:~/Downloads/Taller2Fork-main$ ./taller_procesos 5 archivo1.txt 4 archivo2.txt
```

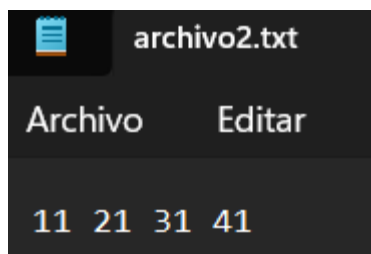
En este caso N1 toma el valor de 5 mientras que N2 de 4. Cabe mencionar que estos valores representan lo siguiente:

- N1: cantidad de enteros en el primer archivo.
- Archivo1: archivo con los enteros del primer arreglo.
- N2: cantidad de enteros en el segundo archivo.
- Archivo2: archivo con los enteros del segundo arreglo.

Los archivos contienen valores enteros separados por espacios, por ejemplo:



Archivo1 3 5 7 9 2



Archivo2 11 21 31 41

El programa debe crear cuatro procesos organizados jerárquicamente y comunicarse entre ellos por medio de pipes, de acuerdo con el siguiente esquema:

- El nieto suma todos los enteros en el arreglo A

- El segundo hijo todos los enteros en el arreglo B
- El primer hijo suma los resultados obtenidos por el nieto y el segundo hijo
- El padre recibe el resultado del primer hijo y lo imprime en la pantalla

3. Descripción de la Implementación

El proyecto consta de tres archivos fuente:

3.1. Funciones.h

Define los prototipos de las funciones:

- void leerArchivo(const char *nombreArchivo, int *arreglo, int N);
- int sumarArreglo(int *arreglo, int N);

3.2. Funciones.c

Implementa las funciones para leer datos y sumar arreglos.

- leerArchivo: abre el archivo indicado, lee N enteros y los almacena en el arreglo dinámico.
- sumarArreglo: recorre el arreglo sumando cada valor y devuelve la suma total.

3.3. taller_procesos.c

Implementa el proceso principal.

Los pasos realizados son:

1. Lectura de argumentos y asignación dinámica de memoria para los arreglos A y B.
2. Lectura de los archivos mediante leerArchivo().
3. Creación de un pipe para comunicación entre procesos.
4. Creación jerárquica de procesos con fork():
 - Primer fork() crea el primer hijo.
 - Dentro del primer hijo, un segundo fork() crea el nieto.
 - Un segundo hijo se genera desde el proceso padre.
5. Comunicación entre procesos:
 - El nieto calcula la suma de A y la retorna mediante exit().
 - El segundo hijo calcula la suma de B y la envía por el pipe.
 - El primer hijo recibe ambos resultados, calcula la suma total y la reenvía al padre.
 - El padre imprime en pantalla las tres sumas finales, la suma del arreglo A y B junto a la suma total en líneas separadas.

6. Liberación de memoria dinámica con `free()`, para liberar los recursos que ya no vamos a usar.

4. Resultados de Ejecución

Comando utilizado:

```
./taller_procesos 5 Archivo1 4 Archivo2
```

Salida esperada:

Suma arreglo A: 26

Suma arreglo B: 104

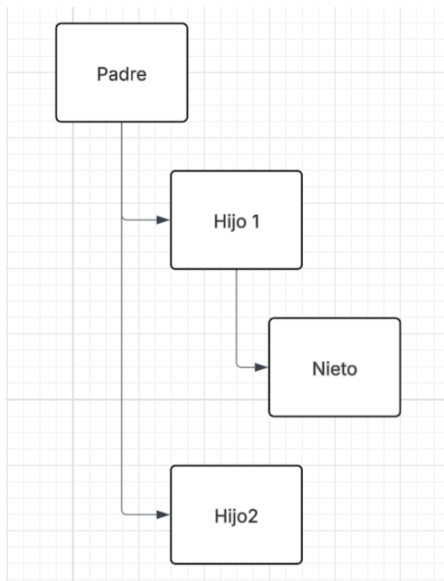
Suma total: 130

Resultado obtenido:

```
estudiante@NGEN154:~/Downloads/Taller2Fork-main$ make
make: 'taller_procesos' is up to date.
estudiante@NGEN154:~/Downloads/Taller2Fork-main$ ./taller_procesos 5 archivo1.txt 4 archivo2.txt
Suma arreglo A: 26
Suma arreglo B: 104
Suma total: 130
estudiante@NGEN154:~/Downloads/Taller2Fork-main$ ls
archivo1.txt  Funciones.c  Makefile      taller_procesos.c
archivo2.txt  Funciones.h  taller_procesos
estudiante@NGEN154:~/Downloads/Taller2Fork-main$
```

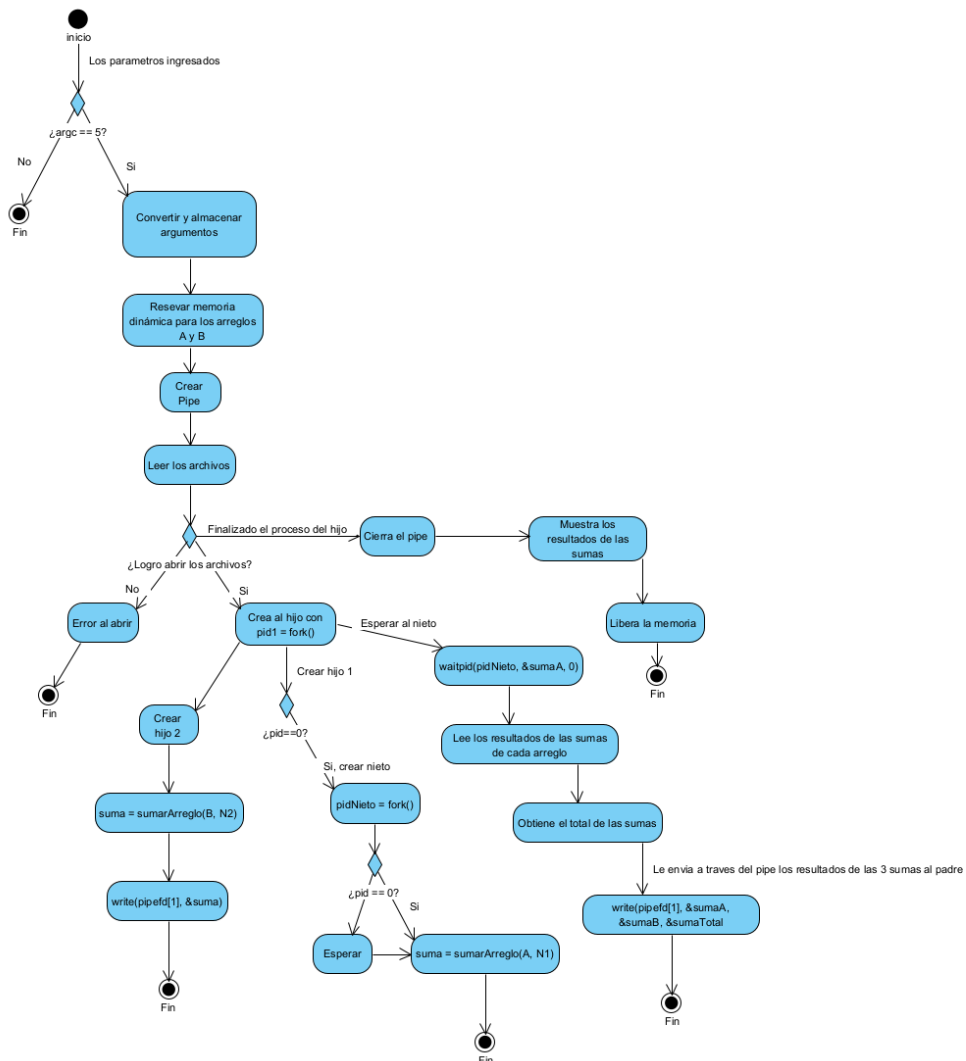
5. Evaluación de Procesos

Visualmente se destaca que el padre tiene dos hijos y que el hijo 1 tiene el nieto.



Cada proceso cumple un rol específico y se sincroniza adecuadamente usando `wait()` y `pipe()`.

El flujo de comunicación es el siguiente:



Cabe mencionar que se manejan diversos finales en el flujo ya que los procesos manejados por el hijo 1, hijo 2 y el nieto cada uno presenta su propio fin. Asimismo, se maneja en su mayoría un flujo exitoso y se aclara que cuando los hijos y nietos alcanzan su final el resultado se transmite a su respectivo padre. De igual manera, se manejan dos modalidades mensajes explicativos en las relaciones y partes de código o únicamente la explicación, pero el diagrama sirve para identificar el flujo del programa.

6. Conclusiones

- Se logró implementar con éxito un sistema multiproceso que demuestra la jerarquía padre-hijo-nieto y la comunicación entre procesos mediante `pipe()`.

- El uso de `fork()` y `wait()` permitió la sincronización de los procesos
- La transferencia de información por descriptores de archivo garantizó un intercambio eficiente de datos entre los procesos.
- La modularización de los archivos permitió un código más ordenado, con un mejor manera de prácticas, más legible y fácil de editar.
- El taller permite fortalecer y aplicar lo trabajado en clase.